`yarg-sync` — the sync client

`yarg-sync` pulls a `yarg-song-server` library into an ordinary local songs folder. **YARG is not modified and does not know the server exists.** It sees files in a folder, which is all it has ever needed to see.

That is the whole point of Phase 2b: shared libraries work today, on stock YARG, on every platform YARG runs on, with no client fork and no risk to anyone's install.

There are now two clients, and this is the one that needs no modified game. The fork mirrors from a server from inside the game (ADR-004), which is nicer to use and requires a build of YARG you made yourself. `yarg-sync` is not superseded by it: it is the path for anyone running a release build, and the two agree byte-for-byte on what they pull, which is how each one checks the other.

```
yarg-sync -server http://pi.local:8080 -songs "%USERPROFILE%\YARG Songs"
```

Flags

Flag	Default	Meaning
`-server`	*(required)*	Base URL of a `yarg-song-server`.
`-songs`	`./songs`	Local folder to sync into.
`-dry-run`	off	Report what would change; write nothing.
`-prune`	**off**	Delete songs the server no longer has.
`-version`		Print version and exit.

Exit status is `0` only if every song the server offered was fetched and verified. A run that fetched some songs and failed on others exits `1`. A sync that silently skipped half a library is exactly the kind of green this project audits other people's scripts for.

What it will and will not touch

The client writes exactly one shape of file:

```
<chart_hash>.sng          # 40 lowercase hex characters, then ".sng"
```

Anything else in that folder is not ours and is never read, renamed, rewritten or deleted — not by a normal run, and not by `-prune`. That includes a loose song folder the player charted themselves, a `.sng` they named by hand, and a file whose name is nearly ours but not quite. `internal/e2e` (`TestThePlayersOwnSongsAreNeverTouched`) snapshots such a folder, syncs, prunes, and compares every byte.

So it is safe to point `-songs` at a folder the player already uses. That was a design requirement, not a happy accident: a sync tool that can eat somebody's own charts is one nobody should run.

`-prune` is off by default and costs an extra request (a second `POST /api/v1/have` with an empty list, to learn the server's full set). Deleting a player's music because the server was briefly reachable but empty is a failure mode worth paying a round trip to avoid.

Why files are named by chart hash

Song identity in this project is `SHA1(chart file bytes)` and nothing else — the same rule the server indexes by and the same rule YARG uses. Naming the file after it makes the local folder self-describing: the client can take inventory by reading directory entries, with no state file to corrupt, lose or disagree with reality.

It also means the client can check the server's work. Every download lands as `<hash>.sng.part`, is opened and scanned locally, and is only renamed into place if the identity re-derived from the bytes matches the one that was asked for. A truncated transfer, a proxy serving something else, or a server bug all fail closed. `TestCorruptDownloadIsRefused` and `TestWrongSongIsRefused` cover it.

When two packages share a chart

Two folders can hold the same chart with different extras — different album art, a different `charter`, an extra stem. Identity is the chart, so the client wants one copy, and the server refuses to guess: `GET /song/{hash}.sng` answers **300 Multiple Choices** with the candidates. The client then picks the lowest package hash, which is arbitrary but *deterministic*: two machines syncing the same server pick the same package.

Picking the same package is half of a shared library. The other half is that the server encodes that package the same way every time, and until 2026-09-06 it did not — `PackDir` drew a random obfuscation mask per call, so two machines that had agreed perfectly about *which* song still

received different files. It was found by running exactly this client on two computers and comparing SHA-256s: 16 of 22 differed. The mask is now derived from the package hash. See `docs/TEST-CORPUS.md`, fourth oracle run.

What the server is trusted for, and what it is not

Choosing a server is not the same as trusting every string it sends. The client trusts the server for **content** — which songs exist, and the bytes of each one — and verifies even that, because a downloaded archive is only renamed into place once its chart hash is re-derived locally. What the server is *not* trusted for is **names**: any string that becomes a path, or part of a request the client then makes, is checked against a shape before it is used.

There are exactly three such strings, and all three are now checked in both clients:

String	Where it arrives	Check	What it could do without one
chart hash	`POST /api/v1/have` → `missing[]`	`^[0-9a-f]{40}$`	Becomes `<hash>.sng` on disk. `filepath.Join` cleans a path but does not confine it, and .NET's `Path.Combine` discards its first argument entirely when the second is rooted — so `../../..` or `C:/Windows/Tasks/x` writes wherever the server said. **A remote arbitrary file write**, and it existed in both clients.
chart hash	again, at the line that builds the path	same	The check does not depend on a caller elsewhere having been careful.
package hash	the `300 Multiple Choices` body	`^[0-9a-f]{16,128}$`	Goes straight into `?package=` on the request that follows. Much smaller: a URL, not a filename, and the chart-hash verification still catches a wrong song. Closed on 2026-09-08 because it was the last server-supplied string used without looking at it.

A server cannot name a file for deletion in either client, and that was checked rather than assumed. `yarg-sync`'s `prune` iterates the LOCAL inventory, which `managedName` has already filtered, and removes only hashes the server does not have — the server's list can subtract from that set, never add to it. The in-game mirror does not prune at all: deciding what to do about a song the server no longer offers is a separate question from getting the ones it does, and answering it wrong costs somebody their library.

The mirror does delete one thing, and auditing that turned up a small mismatch worth fixing. Its sweep of dead partial downloads took **any** name ending `.part`, while the guarantee written a few lines above it says anything not named like ours belongs to the player and is never touched. The mirror folder is one the game owns, so a stranger's `.part` sitting in it is unlikely — but the claim was wrong, and a guarantee that holds "almost always" is not one. The sweep now matches

`^[0-9a-f]{40}\.sng\.part$`, exactly the name the download path can write; anything else counts as the player's and is left alone. The probe puts three such files in the folder — `stranger.part`, `not-a-hash.sng.part`, and one ending `.sng.part.bak` — and asserts all three survive a sync against a server that keeps failing.

A malformed entry is **dropped, not fatal**, in every case: one bad name must not cost a sync of ten thousand good ones. Both clients drop rather than reject, deliberately — a server sending a bad entry still leaves the two clients choosing the same package, which is the entire point of choosing deterministically. If a 300 lists *nothing* usable, that song fails and the reason says why; the client never invents a request.

The pattern for package hashes is a shape check, not a length assertion tied to SHA-256, and it is the same pattern `packcache` enforces server-side before opening a cached archive — the lesson pointed inward as well as outward.

Windows Defender flagged the binary once, then stopped (false positive)

On 2026-09-05 between 20:26 and 20:27, four Defender detections fired on ENG-1 against `cmd/yarg-sync` builds:

```
Trojan:Win32/Bearfoos.A!ml      (ThreatID 2147731250)
```

`go build ./cmd/yarg-sync` failed with "*the file contains a virus or potentially unwanted software*" before it could link, twice; the finished binary was quarantined twice more.

By 21:10 the same day it no longer reproduced, with nothing changed on the machine. Measured rather than assumed:

Test	Result
3 × `go clean -cache` then build to `bin\`, then execute	built, survived, ran
3 × build with a different `-ldflags` version string (3 distinct SHA-256s)	built, survived, ran
New detections across all six	**0**
Defender signature version before and after	identical (`1.459.64.0`)
Exclusions added	**none**

Three distinct hashes ruling it out means this was not a per-file verdict cached clean — the classifier's answer itself changed. `!ml` verdicts are cloud-delivered and get revised upstream; this one was, within the hour.

What to do if it comes back

Re-measure first. The detection above would have justified a permanent Defender exclusion if anyone had acted on it immediately, and that exclusion would still be there now, weakening the machine for a problem that had already evaporated. An `!ml` verdict is provisional by construction.

If it recurs *and persists across a re-test*:

- **Do not add a Defender exclusion**, do not allow-list the threat ID (that allows the whole `Bearfoos.A!ml` family machine-wide, including true positives), and never suggest disabling protection to get a build through.
- **Submit the binary to Microsoft's false-positive form.** Free, and it fixes the verdict for every player who downloads a release, not just for us.
- **Sign the release artifact.** A code-signing certificate is the actual solution for an unsigned executable that strangers download. Note that a self-signed certificate does *not* help here — it does nothing for an ML verdict.

The signing identity is decided: FatalException. This is a personal project and is signed as one; it is never signed as Juniper Design Group, whose certificate would attach a company's name and liability to a personal release.

FatalException is a brand, not a registered legal entity, and that settles the certificate question more than price does. Organization-validated and EV certificates validate an organization; with no entity to validate, the certificate is issued to the *individual* instead. Two consequences, both worth knowing before anything is bought:

- **The publisher string Windows shows will be the developer's validated legal name, not "FatalException".** Individual-validated certificates carry the person. That does not change what this project is *branded* as — the docs and PDFs stay FatalException — but a player clicking through SmartScreen sees a person's name, and it is better to decide that is acceptable now than to discover it after paying.
- Microsoft's own guidance points individual developers at **Azure Artifact Signing** (formerly Trusted Signing), which explicitly supports individuals but **only in the USA and Canada** — which covers us — at roughly $10/month. Outside those countries, or if the individual tier is unsuitable, an OV certificate from a commercial CA is the documented fallback at roughly $150–300/year. Prices and eligibility both move; re-check them at the point of purchase rather than trusting this paragraph.

A **self-signed certificate does not help** and is worth ruling out explicitly: it does nothing for an ML verdict and nothing for SmartScreen, because neither trusts it.

Owning a domain does not change any of this. `badassium.com` is a fine identity for the project and domain control does matter to some validation flows, but a domain is not a legal entity: it does not make an organization-validated certificate available, and it cannot appear as the

publisher in place of a person's name. - It is not a CI problem in any case. The runner is Linux; `release` and the container image are unaffected.

`internal/e2e` still exercises the client in-process rather than executing the built binary. That was originally a workaround for this detection; it is worth keeping on its own merits, since it makes the suite independent of whatever any scanner thinks of a freshly linked executable on any given day.

How this is verified

`internal/syncclient` unit-tests the client against HTTP stubs. Stubs prove the client's own logic, but a stub only ever says what its author believed the server says — it cannot catch the two sides disagreeing about the wire.

`internal/e2e` therefore wires the real `httpapi.Server`, the real `library.Build` and the real `packcache` to the real client and runs whole syncs against a real library on disk:

Test	What would have to break for it to fail
`TestWholeSyncRoundTrip`	Client ends up with every song, each archive readable and each one the song its name claims — identity re-derived from the downloaded bytes.
`TestSecondRunDownloadsNothing`	Sync is idempotent; an up-to-date client transfers zero song bytes.
`TestThePlayersOwnSongsAreNeverTouched`	Nothing unmanaged is altered, including across a `-prune`.
`TestDryRunWritesNothing`	`-dry-run` names every song and writes no entries.
`TestSharedChartHashSyncsOnce`	The 300 Multiple Choices exchange completes and yields one archive.
`TestPackageHashesThatAreNotHashesAreRefused` (unit)	A 300 whose lowest-sorting entry is `../../../../etc/passwd` — or carries `&`, `#` or a space — is skipped for the well-formed one, and a listing with nothing usable fails instead of guessing.

Each was red-proofed on 2026-09-05 by breaking the code it covers and confirming that test, and only that test, failed:

- `managedName` loosened from `^[0-9a-f]{40}\.sng$` to `\.sng$` → `TestThePlayersOwnSongsAreNeverTouched` alone failed.
- the `-dry-run` guard replaced with `if false` → `TestDryRunWritesNothing` alone failed.

- the `packageHash` filter disabled with `if false &&` → `TestPackageHashesThatAreNotHashesAreRefused` alone failed, reporting that the client had asked the server for `../../../../etc/passwd` (2026-09-08).

A test that has never been seen to fail is a test whose meaning is unmeasured.
